

Open-source, markdown-native note and knowledge tools on macOS with professional slide-deck workflows

Executive summary

A small number of open-source “Obsidian-like” tools genuinely meet all of your hard constraints simultaneously: **plain Markdown files as the canonical store**, **good macOS desktop UX**, and a **credible deck-generation path that supports designer templates**. In practice, the winning strategy is to decouple “note/knowledge UI” from “slide build system”: use a note app that treats `.md` as the source of truth, then render decks via **Marp**, **Quarto**, or **Pandoc** so you can apply designer-supplied CSS themes (HTML) or a designer-built **reference PowerPoint** (`.pptx`) template (PPTX). ¹

The most “complete” option for an Obsidian-style knowledge base with automation hooks is **Logseq** (AGPL-3.0), because it combines file-based Markdown graphs (in its “MD” mode), a plugin marketplace, a Reveal.js-backed presentation mode, and—critically—an HTTP API server usable by external tools and AI agents. Its slide story is good but fragmented: built-in presentation mode can be rough, “Slide Reveal” is featureful but paid, and the best “professional” output usually comes from exporting via Pandoc/Quarto into PPTX using a reference template. ²

If your priority is **polished macOS-native UX** and file-based storage with minimal friction, **FSNotes** (MIT) is the cleanest Mac-first choice (Swift, keyboard-centric, dark mode, fast on large note sets, optional git/backups). It has no plugin ecosystem comparable to Obsidian/Logseq, so decks are produced externally (Marp/Quarto/Pandoc), but that’s not a downside if you want a reproducible, template-driven pipeline. ³

For “writer-engineer” workflows (lots of export targets, profiles, templates) **Zettlr** (GPL-3.0) is unusually strong: it is Markdown-native, macOS-friendly (DMG/Homebrew), and built around Pandoc “profiles/defaults files” you can standardise in version control—excellent for repeatable slides to Reveal.js / Beamer / PowerPoint. ⁴

For teams already living in **Nextcloud/ownCloud**, **QOwnNotes** (GPL-2.0-only) provides plain-file Markdown plus a scripting engine with an online script repository—helpful for automation and file-based AI workflows, though the macOS UX is more “cross-platform Qt” than “native Mac”. ⁵

Two newer, power-user-tempting options are **Tangent Notes** (Apache-2.0) and **Yank Note** (AGPL-3.0). Tangent is local-first and Markdown-file-based with a distinctive sliding-panel “thread” UI and CSS customisation, but some lifecycle features (e.g., exports) are still emerging. ⁶ Yank Note is a highly hackable Markdown editor that explicitly saves as local Markdown and includes Reveal.js presentation support and plugins—but it *also* explicitly trades away safety constraints (arbitrary command/file access), which matters if you’re going to point it at untrusted repos. ⁷

Evaluation criteria and methodology

Selection and analysis followed your constraints literally:

Plain Markdown as **native** storage (not “exportable”), macOS desktop availability, and open-source licensing were treated as gating requirements. Evidence was taken primarily from upstream docs, Git repositories, and official project sites; secondary sources were used only where upstream docs were silent or to provide UX/operational context. ⁸

Slide capability was evaluated in two layers:

- In-app slide modes (often Reveal.js-based) and their practical limitations.
- External “build systems” (Marp / Quarto / Pandoc) because these are the realistic route to designer-quality output and template reuse (CSS themes, LaTeX Beamer themes, or PowerPoint reference templates). ⁹

Programmatic friendliness was assessed for: file layout stability, metadata conventions, plugin/extension APIs, CLI hooks, and local automation affordances (e.g., HTTP API server, scripting engines, reproducible export configs). ¹⁰

Comparison table

App	Licence	Native Markdown storage	macOS UX rating (1–5)	Slide export options	Template availability	Programmatic friendliness	Recommend use case
FSNotes ¹¹	MIT ¹²	Yes ¹²	5	External: Marp / Quarto / Pandoc to Reveal.js, Beamer, PPTX; can also feed Deckset ¹³	Strong via Marp CSS themes; Quarto branding + reference PPTX; Pandoc reference PPTX; Deckset themes ¹⁴	High (plain files + optional git) ¹²	“Native Mac vault” + deterministic deck builds

App	Licence	Native Markdown storage	macOS UX rating (1–5)	Slide export options	Template availability	Programmatic friendliness	Recommend use case
Logseq ¹⁵	AGPL-3.0 ¹⁶	Yes (MD graphs) ¹⁷	4	Built-in Reveal.js presentation mode; plugins: Slide Reveal (Reveal.js + PDF), Pandoc export to PPTX; external Marp/ Quarto/ Pandoc ¹⁸	Reveal themes; Slide Reveal theme list; PPTX reference templates via Pandoc/ Quarto ¹⁹	Very high (HTTP API + plugin API + file store), but MD/DB split complicates ²⁰	Obsidian-like graph + automation + slide extraction from notes
Zettlr ²¹	GPL-3.0 ²²	Yes ²³	4	Built-in Reveal.js export; Pandoc profiles can target Beamer / PowerPoint ²⁴	Strong via Pandoc templates/ reference PPTX and Zettlr profiles ²⁵	High (profiles are YAML; file/ workspace model) ²⁶	Publication workflow + repeatable, template-driven slide exports
QOwnNotes ²⁷	GPL-2.0-only ²⁸	Yes ²⁹	3	Mostly external (Marp/ Quarto/ Pandoc/ Deckset); can automate via scripting ³⁰	External template ecosystems; light in-app styling via scripts/ preview ³¹	High (script engine + script repo) ³²	Nextcloud-ce notes with scriptable automations
Tangent Notes ³³	Apache-2.0 ³⁴	Yes (Markdown “ish”) ³⁴	4	External: Marp/ Quarto/ Pandoc/ Deckset; export features appear on roadmap ³⁵	External template ecosystems; Tangent supports custom CSS for note views ³⁶	Medium (plain files; no public API shown in upstream docs captured) ³⁷	Navigation-first thinking/writing with slide build as a separate step

App	Licence	Native Markdown storage	macOS UX rating (1–5)	Slide export options	Template availability	Programmatic friendliness	Recommend use case
Yank Note 38	AGPL-3.0 7	Yes 7	4	Built-in Reveal.js support; external Marp/ Quarto/ Pandoc/ Deckset also works 39	Reveal themes; plus external template ecosystems; plugins can extend 40	High but risky: explicitly “sacrifices security protection” for extensibility 41	“Notes as an executable project” + slid mode, when y control the content source

Candidate deep dives

FSNotes

Executive summary

FSNotes is the most Mac-native option in this set: a Swift notes manager that is explicitly “Markdown-first,” stores notes as plain files, scales to very large collections, and is designed to be keyboard-centric with dark mode. Its deck story is intentionally “bring your own build system”: use Marp/Quarto/Pandoc (and optionally Deckset) against the same `.md` source. 42

Key attributes

FSNotes’ public repository states MIT licensing, identifies itself as “Markdown-first,” and lists sync via iCloud Drive or Dropbox plus optional Git versioning/backups. 12
macOS availability is via the macOS App Store per the project README, and it runs as a native Swift application. 12

Native Markdown features

Based on the upstream feature list, FSNotes supports tags, `[[double bracket]]` links, code block syntax highlighting, inline images, Mermaid, MathJax, external editor live sync, encryption, and dark mode. 12

Plugin system and integrations

FSNotes does not present a first-class plugin marketplace. The integration story is file-based (open folder, use external editor/tools) plus optional Git integration. 12

Sync options

iCloud Drive and Dropbox are explicitly listed; Git versioning is offered as an option for versioning/backups. 12

Slide export workflows and frameworks

FSNotes itself does not claim a built-in Reveal.js/Marp/Remark mode; you produce slides by rendering Markdown externally:

- **Marp:** Markdown → HTML/PDF/PPTX with CSS theme support. 43

- **Quarto:** Markdown/QMD → Reveal.js, Beamer (PDF), PPTX; supports reference PPTX templates and branding. ⁴⁴
- **Pandoc:** Markdown → Reveal.js/Beamer/PPTX and supports reference PPTX templates. ⁴⁵
- **Deckset:** Markdown file → native macOS presentation app with theming. ⁴⁶

Designer templates / marketplaces

Your strongest “designer template” pathway is **PPTX reference templates** (Pandoc/Quarto) because designers already work in PowerPoint/Keynote-style tools. Both Pandoc and Quarto document `reference-doc` support for PPTX styling. ⁴⁷

If you’re staying in HTML slides, Marp’s CSS theming is the intended customisation mechanism. ⁴⁸

If you want theme marketplaces, Deckset supports theming and command-based overrides, and there are open theme repos. ⁴⁹

Steps to produce a pro-quality deck from Markdown

A pragmatic “designer-friendly” approach is PPTX output using a reference template:

```
# 1) Write slides.md in FSNotes (plain Markdown)
# 2) Export to PPTX using Marp (fast)...
npx @marp-team/marp-cli@latest slides.md --pptx -o deck.pptx

# ...or export to PPTX using Pandoc with a designer reference PPTX
pandoc slides.md -t pptx --reference-doc=brand-template.pptx -o deck.pptx

# ...or use Quarto for more structure/metadata
quarto render slides.qmd --to pptx
```

Marp CLI explicitly supports PPTX/PDF/HTML output. ⁵⁰ Pandoc documents PPTX customisation via `--reference-doc`. ⁵¹ Quarto documents PPTX output plus reference templates and branding options. ⁵²

UX notes for macOS

FSNotes positions itself as fast/lightweight, keyboard-centric, and dark-mode aware, including smooth operation with “10k+ files”. ¹²

Programmatic access notes

This is essentially optimal for code-model workflows: notes are files; links and tags are in text; and Git is a first-class option. That makes it straightforward to implement “model writes Markdown → CI builds slides → reviewer opens output”. ⁵³

Limitations

No upstream-documented plugin marketplace means you won’t get Obsidian-style in-app “slide plugins”; you’re expected to go through external renderers. ¹²

Logseq

Executive summary

Logseq is the closest open-source analogue to Obsidian’s “local graph + plugins” experience. It works on local plain-text Markdown/Org files (for its MD workflow), has a plugin ecosystem, supports Reveal.js-style presentation mode, and offers an HTTP API server and plugin APIs that make it unusually amenable to automation and AI agents. The trade-off is format complexity: Logseq’s block-based

structure and custom syntax can leak into exports, and Logseq's newer DB mode introduces a parallel storage/sync model you must manage deliberately if "plain Markdown files" is non-negotiable. ⁵⁴

Key attributes

The main repository states AGPL-3.0 licensing and provides the project as a desktop/mobile platform with plugin APIs documented externally. ⁵⁵

macOS availability is via standard downloads (GitHub releases are explicitly linked from the repo) and the app is also listed in the Apple App Store as a "local, open-source knowledge base" working on Markdown/Org files. ⁵⁶

Native Markdown features

The App Store description enumerates backlinking, knowledge graph, page/block references & embeds, tasks, and integrations. ⁵⁷

The core README emphasises Markdown/Org support and a plugin/theme ecosystem. ¹⁶

Plugin system

Logseq has an official marketplace listing plugins, and it maintains plugin API documentation. ⁵⁸

Sync options

For MD graphs, Logseq staff/posts state that any file sync service can sync the graph because notes/settings are plain files (but this is distinct from "conflict-free" sync). ⁵⁹

For DB graphs, a forum answer warns that DB mode is intended to work with Logseq Sync and later real-time collaboration, and recommends staying on Markdown/Org for Git/Syncthing style file sync. ⁶⁰

Logseq's own materials describe DB mode's "RTC" collaboration/sync approach and position "sync/real-time collaboration/publishing" as hosted services. ⁶¹

Slide export workflows and frameworks

Built-in presentation mode (Reveal.js-backed)

Community discussions describe Logseq presentation mode as obeying the page hierarchy and treating each bullet as a slide by default (a frequent complaint when building "real" decks). ⁶²

There is also evidence of Reveal.js being embedded in the presentation feature (e.g., issue threads referencing Reveal.js behaviour). ⁶³

Styling often requires custom CSS; community posts show concrete CSS fixes for practical presentation defects. ⁶⁴

Slide Reveal plugin (Reveal.js)

A dedicated plugin, "Logseq Slide Reveal", provides a Reveal.js-based slide workflow with themes and PDF export. Its docs explicitly describe configuration (themes) and PDF export via a print rendering mode. ⁶⁵

The corresponding repository calls out that it is a paid plugin (free users can only view a limited number of slides). ⁶⁶

Pandoc export plugin (PPTX/Docx/HTML/LaTeX)

The `logseq-pandoc-plugin` repository states it converts pages/blocks to DOCX, PPTX, HTML, and LaTeX, preserves formatting/structure, and supports citations. It also documents a key limitation: filters are not supported by `pandoc-wasm`, so it offers "copy the required shell command" for local execution instead. ⁶⁷

Separately, community discussion notes that Logseq-specific Markdown idiosyncrasies can leak into exports, which directly impacts “deck quality” unless you normalise content. ⁶⁸

Designer templates / marketplaces

- Reveal.js themes exist and are selectable in Slide Reveal (theme names are listed). ⁶⁹
- For PPTX output, the designer-template route is again “reference PPTX”: Pandoc documents PPTX customisation via `--reference-doc`, and Quarto documents `reference-doc` for PPTX output. ⁴⁷

Steps to produce a pro-quality deck from Markdown

Two “tell it like it is” recipes:

Recipe A: In-Logseq quick slides (fast, less controllable)

- Write a page with a deliberate outline structure.
- Use built-in presentation mode (bullets → slides) and patch CSS where needed. ⁷⁰

Recipe B: Professional PPTX via Pandoc (best for designer templates)

- Write a dedicated “deck page” in Logseq with minimal Logseq-specific syntax (avoid block embeds/refs that won’t export cleanly).
- Export to `.md` /intermediate and render to PPTX with a designer reference template.

```
# Example: render a deck.md (exported/maintained alongside the graph) to PPTX
pandoc deck.md -t pptx --reference-doc=brand-template.pptx -o deck.pptx
```

Pandoc explicitly supports PPTX output and styling via reference documents. ⁷¹

UX notes for macOS

The project positions itself as a local-first knowledge base and is distributed as a desktop app with macOS availability. ⁷²

Direct evidence of polished macOS-native UX (AppKit-level) is not presented in upstream docs; in practice it behaves like a cross-platform desktop app, and the presentation mode quality has been criticised as needing patching. ⁶⁴

Programmatic access notes

Logseq is unusually strong here:

- Plugin APIs are publicly documented. ⁷³
- External applications can interact via the HTTP API server; multiple integration docs instruct enabling the HTTP API server, setting/generating tokens, and requiring Logseq to be running. ⁷⁴
- This enables explicit AI-agent workflows (e.g., MCP servers that connect to the local HTTP API). ⁷⁵

Limitations

- Export fidelity: Logseq-specific syntax and block semantics can degrade Pandoc/slide exports unless your content is authored with export in mind. ⁷⁶
- Storage bifurcation: DB mode moves away from “plain Markdown files” as the canonical store (SQLite DB + assets on disk), so if your constraint is strict, you must stick to MD graphs and treat DB features as

out-of-scope. ⁷⁷

- Slide Reveal is paid; the best “deck-like” in-app experience may not be fully open/free. ⁶⁶

Zettlr

Executive summary

Zettlr is less a “graph PKM” and more a “publication workbench”: it’s Markdown-native, export-centric, and built around configurable Pandoc profiles (“defaults files”). That makes it excellent for professional slide pipelines—especially when your goal is “one Markdown source → multiple outputs (Reveal.js HTML, Beamer PDF, PowerPoint PPTX) with consistent templates”. ⁷⁸

Key attributes

Zettlr is GPL-3.0 licensed. ²²

macOS distribution is via DMG (drag to Applications) and via Homebrew cask. ⁷⁹

Native Markdown features

Zettlr is explicitly positioned as a Markdown editor that can export via Pandoc, supports internal links and “Zettelkasten” workflows, and exposes export profiles through its Assets Manager. ⁸⁰

Plugin system

Zettlr’s upstream docs do not describe a plugin marketplace in the way Logseq does. Instead, extensibility is via export/import profiles, templates, snippets, and (per independent review) even custom export commands beyond Pandoc. ⁸⁰

Sync options

Zettlr’s model is file/workspace-based; upstream docs focus on installation and exporting, not a proprietary sync layer. In practice, any file sync (git, Dropbox, etc.) works because the files are normal documents; upstream source describes workspaces and file mapping. ⁸¹

Slide export workflows and frameworks

Zettlr’s documentation states it can export Markdown directly as presentations using Reveal.js, and that it ships with a profile for presentations that you can customise. ⁸²

The project site further claims you can create profiles for Beamer and PowerPoint as well. ⁸³

Zettlr’s “defaults files” documentation explains that profiles are YAML files controlling Pandoc export behaviour and are editable via the Assets Manager. ⁸⁴

Pandoc itself confirms it can produce Reveal.js HTML slides, Beamer PDF slides, and PowerPoint slide shows. ⁸⁵

Designer templates / marketplaces

This is where Zettlr shines because “template-driven export” is its core identity:

- Pandoc supports PPTX customisation via `--reference-doc`. ⁸⁶
- Zettlr profiles are the place to encode those flags and parameters for one-click export. ⁸⁷

Steps to produce a pro-quality deck from Markdown

Reveal.js (HTML deck) from within Zettlr

Zettlr’s presentations guide describes using Reveal.js export and mentions a built-in export profile. ⁸²

PowerPoint (PPTX) using a designer reference template

Create or copy a Zettlr defaults file/profile with `writer: pptx`, then set a reference template.

Example `export.pptx.brand.yaml` profile concept:

```
# In Zettlr Assets Manager -> Defaults Files / Profiles (conceptual example)
writer: pptx
reference-doc: /absolute/path/to/brand-template.pptx
slide-level: 2
```

Then export using that profile from the Export dialog.

Zettlr documents that profiles/defaults files are YAML controlling Pandoc export. ⁸⁷ Quarto and Pandoc both document PPTX reference templates; the same concept applies when Zettlr delegates to Pandoc. ⁸⁸

UX notes for macOS

Upstream installation instructions are conventional DMG drag-install and Homebrew cask; its docs also emphasise an integrated manual and preference system. ⁸⁹

Programmatic access notes

- The primary interface is the filesystem (folders/workspaces/projects). ⁸¹
- Export profiles are YAML files; this is good for automation because you can generate/update them programmatically and commit them to version control to standardise exports across a team. ⁸⁴

Limitations

- Zettlr's slide UX is export-oriented rather than "live slide mode while you brainstorm" (it's not a PKM graph tool first). ⁹⁰
- If you rely on PPTX reference templates: Pandoc has open issues showing PPTX templates can become "corrupt" depending on how they are edited/saved (especially via non-PowerPoint editors), which is a real operational foot-gun in branded pipelines. ⁹¹

QOwnNotes

Executive summary

QOwnNotes is a conservative but durable "plain Markdown files + sync via Nextcloud" solution. Where it gets interesting for your use case is the scripting engine and script repository: you can build automation hooks that manipulate notes, run exports, and integrate AI-adjacent workflows while keeping everything file-based. ⁹²

Key attributes

Upstream positioning: "Free open source plain-text file markdown note-taking with Nextcloud / ownCloud integration," with notes stored as plain Markdown files and sync via Nextcloud. ²⁹

macOS install is DMG (from GitHub Releases) with drag-to-Applications; the current universal binary requires macOS 13 (Ventura) and there is a Qt5 legacy DMG. ⁹³

Licensing is described as GPL in multiple sources, including a "GPL-2.0-only" classification in an encyclopaedic summary. ²⁸

Native Markdown features

QOwnNotes emphasises Markdown support, to-do management, local file storage, and Nextcloud/ownCloud interoperability. ⁹⁴

Plugin system / automation

QOwnNotes' "scripting" is effectively its plugin system: scripts are mostly JavaScript inside Qt QML files and can be added via settings. ⁹⁵

There is an official script repository you can access from within the app. ⁹⁶

Its scripting docs also describe hooks for integrating OpenAI-compatible backends via scripts—useful if you want tightly controlled "local tool calls" behaviour. ⁹⁷

Sync options

The official site explicitly suggests syncing notes via Nextcloud. ²⁹

Slide export workflows and frameworks

No upstream evidence in the captured sources indicates first-class built-in slide frameworks (Reveal.js/Marp/Remark). In practice, because notes are plain files, you use external renderers (Marp/Quarto/Pandoc) or pass a Markdown file into Deckset. ⁹⁸

Designer templates / marketplaces

You inherit the same template ecosystems described later in this report (reference PPTX templates, CSS themes, Deckset themes). ⁹⁹

Steps to produce a pro-quality deck from Markdown

Given a `deck.md` located inside your QOwnNotes notes folder:

```
# Fast HTML/PDF/PPTX slides via Marp
npx @marp-team/marp-cli@latest deck.md --pptx -o deck.pptx

# PPTX using a designer template via Pandoc
pandoc deck.md -t pptx --reference-doc=brand-template.pptx -o deck.pptx

# Reveal.js HTML deck via Pandoc (then host statically if needed)
pandoc deck.md -t revealjs -s -o deck.html
```

Marp CLI's output formats are explicitly documented. ⁵⁰ Pandoc's slide-show targets and PPTX template support are documented. ⁷¹

UX notes for macOS

Installation is DMG drag-install; there are explicit macOS Gatekeeper/unquarantine notes and a built-in updater in the install docs. ¹⁰⁰

The Homebrew cask listing indicates deprecation in the cask tap (with a future disable date), which is operationally relevant if you standardise installs via brew. ¹⁰¹

Programmatic access notes

Because each note is a file, code models can operate on it directly. On top of that, QOwnNotes scripting provides API surface to implement "export current note," "insert front matter," "normalise links," etc. The

docs show scripts can present file-save dialogs and reference examples like “export notes as one HTML”.

102

Limitations

- UX polish is bounded by cross-platform Qt conventions; it is not a Mac-native “system” app. 93
- If you want Obsidian-like graph/PKM affordances, those aren’t the primary story in the upstream positioning. 29

Tangent Notes

Executive summary

Tangent Notes is a local-first Markdown notes app with a distinctive navigation-as-context design: sliding “thread” panels and a map view. It is open source under Apache-2.0 and explicitly states that it reads/writes Markdown files directly and supports syncing via “whatever you prefer” (git/iCloud/Drive/Dropbox). Its slide pipeline is therefore external (Marp/Quarto/Pandoc/Deckset), and some export functionality appears on the project roadmap rather than being a mature, documented feature. 103

Key attributes

The Tangent monorepo is Apache-2.0 licensed and describes Tangent as a local PKM tool inspired by Obsidian, using wiki links and a slightly customised Markdown syntax. 104

The official site states that Tangent works with local Markdown files, is open source under Apache-2.0, and is available for Mac/Windows/Linux. 105

The same site claims it supports syncing/versioning via git and common cloud drives. 106

An influence explicitly cited is Andy Matuschak 107’s notes (sliding-panel inspiration). 104

Native Markdown features

Tangent supports embedded media (images, PDFs, audio/video), link previews, custom query language, and customisable note views per its upstream README. 104

Its features page adds support for custom CSS and creation rules (templated names/folders). 106

Plugin system

Upstream docs captured here do not describe a plugin API/marketplace. Tangent’s extensibility story in official materials is primarily theming and CSS customisation. 108

Sync options

Explicitly “use whatever system you prefer,” including git/iCloud/Drive/Dropbox. 106

Slide export workflows and frameworks

No upstream slide mode is documented in the captured sources. Treat Tangent as a filesystem-native editor and push slides through external renderers to reach Reveal.js/Beamer/PPTX/Deckset. 109

Designer templates / marketplaces

Same as other file-based tools: PPTX reference templates (Pandoc/Quarto), CSS themes (Marp/Reveal.js), Deckset themes. 110

Steps to produce a pro-quality deck from Markdown

A simple “deck note” pattern:

```
# Put deck.md inside your Tangent workspace folder
# Render with Quarto (Reveal.js HTML)
quarto render deck.qmd --to revealjs

# Render to PPTX using a reference template
quarto render deck.qmd --to pptx
# or Pandoc
pandoc deck.md -t pptx --reference-doc=brand-template.pptx -o deck.pptx
```

Quarto documents Reveal.js and PPTX outputs for presentations, including PPTX templates via `reference-doc`. ⁴⁴ Pandoc documents PPTX reference templates. ⁸⁶

UX notes for macOS

Official site positions it as a desktop app built for Mac/Windows/Linux; upstream README notes an Electron-based app with developer tooling accessible (developer console, command palette). ³⁷

The official roadmap lists export-related items (e.g., export notes to HTML/simplified Markdown) as future work, which is a maturity signal. ¹¹¹

Programmatic access notes

Programmatic access is file-based; Tangent reads/writes Markdown files directly, making “AI agent operating on a repo” feasible. ³⁷

Caveat: Tangent uses “slightly customised Markdown syntax,” so if you build slides with strict parsers, you may need a normalisation pass (e.g., convert custom constructs to CommonMark/Pandoc-friendly Markdown). ¹⁰⁴

Limitations

- Some export capabilities appear as roadmap items rather than stable, documented features. ¹¹¹
- Custom Markdown flavour can reduce interoperability unless you constrain your authoring conventions. ¹⁰⁴

Yank Note

Executive summary

Yank Note is a “Markdown IDE”: local Markdown files, extensible plugins, built-in version control and many embedded features—including Reveal.js presentations. It also explicitly warns that it sacrifices security protections (including arbitrary command execution and file read/write) to be extensible. That makes it powerful for controlled, code-model-driven workflows on trusted content, and a bad idea as a general “open random vaults” tool. ⁷

Key attributes

The upstream repository provides AGPL-3.0 licensing, states that data is saved as local Markdown files, supports plugins, and includes Reveal.js functionality; it also provides macOS builds (arm64 and x64) in its release notes section. ⁴¹

The project site reiterates open-source AGPL-3.0 licensing and local storage. ¹¹²

Native Markdown features

The README highlight list includes local Markdown storage, plugin extension, encryption, embedded diagrams (Mermaid), mind map views, attachments, and more. ⁴¹

Plugin system

Yank Note explicitly supports user-written plugins (located in a `home directory/plugins` path per README). ⁴¹

Sync options

The upstream emphasis is “local files + version control”; it does not claim a proprietary sync service in the captured sources. ⁷

Slide export workflows and frameworks

Upstream materials confirm Reveal.js support. ⁴¹

Because content is local Markdown, external renderers (Marp/Quarto/Pandoc) are also straightforward if you need PPTX templates or Beamer output. ⁹

Designer templates / marketplaces

- Reveal.js theming is available via the Reveal ecosystem and plugins. ¹¹³

- PPTX reference templates are available via Pandoc/Quarto when you decide you need designer-grade branding. ⁴⁷

Steps to produce a pro-quality deck from Markdown

If you keep a `deck.md` file inside your Yank Note workspace, this is a robust “output-first” workflow:

```
# Branded PPTX using a designer template via Pandoc
pandoc deck.md -t pptx --reference-doc=brand-template.pptx -o deck.pptx

# Or produce a web deck (Reveal.js HTML) via Pandoc
pandoc deck.md -t revealjs -s -o deck.html
```

Pandoc documents Reveal.js + PPTX slide outputs. ⁷¹

UX notes for macOS

The upstream release notes list dedicated macOS builds (arm64/x64). ⁴¹

Programmatic access notes

File-based workflows are excellent here. Additionally, Yank Note’s declared “arbitrary file read/write and command execution” capability makes it possible to build extremely direct automations and “agent actions”—but that power is inseparable from the security risk. ⁴¹

Limitations

The upstream README explicitly warns that extensibility comes from sacrificing security protections and urges caution with untrusted Markdown. That is a hard constraint in many real environments (even if you said “no corporate constraints,” it affects your own threat model). ⁴¹

Slide-deck pipelines and template ecosystems

A realistic workflow model

Most note apps (including Obsidian-like ones) do not contain a “designer template system” on par with PowerPoint/Keynote. The professional move is to treat Markdown as *content*, and use a slide compiler that has first-class theming/template support:

- **HTML slide decks:** Reveal.js-based output (directly or via Pandoc/Quarto) + CSS themes/plugins. ¹¹⁴
- **PPTX decks:** Pandoc/Quarto → PPTX using `reference-doc` as the designer template. ⁴⁷
- **PDF decks:** Beamer via Pandoc/Quarto with LaTeX themes; good for technical talks and print-quality output. ¹¹⁵
- **Markdown-to-presentation apps:** Deckset is a Mac app that turns Markdown into themed presentations with custom theming controls. ⁴⁶
- **Markdown slide engines:** Marp converts Markdown to HTML/PDF/PPTX and supports CSS themes. ⁴⁸

Mermaid workflow flowchart

```
graph LR
    A["Markdown note files<br/>deck.md / slides.qmd"] --> B["Where do slides render?"]
    B -->|In-app mode| C["Reveal.js mode<br/>e.g. Logseq presentation"]
    B -->|CLI build| D["Slide compiler<br/>Marp / Quarto / Pandoc"]
    C --> E["Theme / CSS tweaks<br/>app theme or custom CSS"]
    D --> F["Apply designer template"]
    F -->|HTML slides| G["CSS theme pack<br/>Reveal.js / Marp theme"]
    F -->|PowerPoint| H["reference PPTX template<br/>brand-template.pptx"]
    F -->|PDF| I["Beamer theme / LaTeX template"]
    E --> J["Final deck<br/>HTML / PDF / PPTX"]
    G --> J
    H --> J
    I --> J
```

Reveal.js: strong for “web-native” decks

Reveal.js is explicitly an open-source HTML presentation framework; it supports Markdown authoring and a plugin ecosystem (Markdown, code highlighting, speaker notes, etc.). ¹¹⁶

Where Reveal.js shines: interactive embeds, precision CSS, and a single portable HTML artefact. Where it hurts: if you need to hand over editable slides to non-technical colleagues, you often end up back at PPTX (hence the PPTX reference-template pathway). ¹¹⁷

Practical “pro deck” tip: treat Reveal.js decks as “web products”: keep a `theme.css`, lock fonts, host assets locally, and standardise speaker notes (Reveal’s speaker notes plugin is a first-class feature). ¹¹⁸

Marp: the cleanest Markdown→PPTX bridge

Marp positions itself as a Markdown presentation ecosystem with built-in themes and CSS theming, and it explicitly supports exporting to HTML/PDF/PowerPoint. ¹¹⁹

Marp CLI's README is unusually actionable: it shows direct commands for HTML/PDF/PPTX conversion and supports watch/server modes for iterative deck work. ⁵⁰

If you want tight integration inside a code-editor workflow, the Marp for VS Code extension integrates Marp CLI export commands directly. ¹²⁰

Template strategy: CSS theme packs are your “designer templates” for Marp HTML/PDF; for PPTX branding, you may still prefer Pandoc/Quarto if you have a hard designer-provided PPTX master (Marp PPTX output is good, but PowerPoint templating is not its core “design surface”). ¹²¹

Quarto: structured docs + presentations with branding

Quarto explicitly supports presentation formats including Reveal.js (HTML), PPTX (PowerPoint), and Beamer (LaTeX/PDF). ¹²²

Its PowerPoint reference docs describe the `reference-doc` option for styling and a `brand` mechanism for branding information. ¹²³

Quarto's PowerPoint documentation also claims support for speaker notes and common presentation layout features, plus rendering with custom PPTX templates. ¹²⁴

For “professional decks with designer templates,” Quarto is often the sweet spot: you can keep a repo with `slides.qmd`, a `brand.yml` /assets, a `reference.pptx`, and deterministic builds. ¹²⁵

Pandoc: the universal fallback (and sometimes the best choice)

Pandoc documents that it can produce HTML+JS slide presentations (including Reveal.js) and also PPTX and Beamer. ⁸⁵

It also explicitly documents that PPTX output can be customised using `--reference-doc`, and provides examples including background images via reference layouts. ⁵¹

Hard truth: PPTX reference templates can be fragile. Pandoc issue reports show “PowerPoint template warning” and corruption behaviour depending on how the template is edited/saved, which means you should validate your “designer master” toolchain and lock the template creation/edit process. ⁹¹

Deckset: worth knowing even though it's not open source

Deckset is a macOS app that turns Markdown files into presentations and has theming/custom theming controls documented in its help site. ⁴⁶

There are also open repositories containing Deckset themes/templates (useful starting points or internal theme packs). ¹²⁶

Deckset can be valuable when you want a designer-polished look quickly and you accept the fact the slide compiler is a proprietary app—your notes and slide sources remain plain Markdown files, so the lock-in is lower than “binary slide file only.” ⁴⁶

Remark: minimal, hackable, but CSS-heavy

Remark is an in-browser Markdown-driven slideshow tool aimed at people comfortable with HTML/CSS. ¹²⁷

It can produce beautiful results, but you'll be designing your own theme system or adopting a template repo—this is great for engineering teams and less great if you want a turnkey “designer marketplace”. ¹²⁷

Programmatic access for code-model workflows

File-based workflows are the baseline

All candidates here ultimately succeed or fail on whether Markdown files are **the canonical artefact**. FSNotes, QOwnNotes, Tangent Notes, and Yank Note explicitly centre local Markdown storage; Logseq (MD graphs) does as well, though its DB mode introduces a different “source of truth”. ¹²⁸

For Claude/Codex-style workflows, file-based storage enables:

- deterministic diffs and review (git PRs against `.md`)
- automated linting/normalisation (Markdown lint, link checks)
- automatic slide builds (CI builds HTML/PDF/PPTX)
- prompt-driven content transforms (summarise notes → build deck) without needing to automate GUI state.

Marp CLI’s docs and Quarto’s docs both support CLI rendering, which is what you want for non-interactive build systems. ¹²⁹

API-driven workflows (Logseq stands out)

Logseq is unusual among note tools because it has a local HTTP API server (token-based) and a documented plugin API. Multiple integration docs show the operational truth: Logseq must be running, you enable “HTTP APIs server,” and you manage an auth token. ¹³⁰

This is exactly the kind of interface that code models can use safely via a constrained intermediary (e.g., an agent tool that only exposes “search”, “create page”, “append block”). ⁷⁵

Automation hooks by tool

- **Logseq**: best-in-class for API automation (HTTP API + plugin API), plus plugin marketplace. ¹³¹
- **QOwnNotes**: strong via built-in scripting (JS/QML), online script repository, and documented hooks (including AI backend hooks). ¹³²
- **Zettlr**: strong via “profiles as YAML” and export automation; profiles can be generated/managed as code. ⁸⁴
- **FSNotes / Tangent Notes**: primarily filesystem workflows; automation via external tools rather than internal APIs. ¹³³
- **Yank Note**: automation via plugins but explicitly dangerous by design when pointed at untrusted content. ⁴¹

Limitations and forward-looking recommendations

Interoperability is mostly about Markdown flavour discipline

Your slide build quality will be limited less by the note app and more by how “standard” your Markdown stays. Logseq and Tangent explicitly introduce custom semantics (block structures, customised Markdown), and user reports show that non-standard constructs can survive into exports and look ugly.

¹³⁴

The forward-looking solution is to adopt a **“deck subset” authoring convention**:

- Keep “deck notes” in a dedicated folder (e.g., `/decks`) with restricted syntax (CommonMark + a small set of supported extensions).
- Run a normaliser step (e.g., convert wiki links / block refs to standard links) before rendering.
- Store template artefacts (CSS theme, `reference.pptx`, fonts/assets) in the repo.

This reduces both export friction and AI-agent error rates. ¹³⁵

PPTX reference templates are the most “designer-compatible,” but brittle

If your goal is “professional slide deck with designer-made templates,” the most organisation-compatible path is: designer builds a PowerPoint template → CI produces PPTX from Markdown using `reference-doc`. Pandoc and Quarto support this explicitly. ⁴⁷

But Pandoc issue reports show template corruption can occur depending on how templates are modified and saved, so you should standardise “template authoring rules” and validate output in an automated test (open/repair checks). ⁹¹

The “native macOS UX” bar is met primarily by FSNotes

If you care about macOS feel (performance, keyboard ergonomics, native UI idioms), FSNotes is the standout because it is explicitly a Swift macOS/iOS app and emphasises keyboard-centric usage, dark mode, and large-vault performance. ¹²

Everything else here is effectively cross-platform UI, even if it runs “well enough” on macOS.

Logseq’s MD vs DB trajectory matters

Logseq’s DB mode (SQLite + assets on disk) and collaboration/sync roadmap is moving the product toward a more app-managed data layer and hosted services (RTC/sync/real-time collaboration/publishing). If file-native Markdown is your non-negotiable base, treat MD mode as the only supported posture and watch DB mode as a separate product line. ¹³⁶

Concrete “best fit” recommendations

- **Best overall for Obsidian-like workflows + automation + slides:** Logseq (MD graphs) + Pandoc or Quarto for branded PPTX builds; use in-app presentation mode only for quick drafts. ¹³⁷
- **Best Mac-native note vault with pro slide builds:** FSNotes + Marp CLI (fast decks) or Quarto/Pandoc (branded PPTX). ¹³⁸
- **Best for “profiles, templates, repeatable exports”:** Zettlr + Pandoc reference PPTX (and/or Beamer themes), with profiles committed alongside project docs. ¹³⁹
- **Best when Nextcloud is your backbone:** QOwnNotes + script automation + external slide renderer (Quarto/Pandoc/Marp). ¹⁴⁰
- **Best “experimental UI / thought-navigation”:** Tangent Notes, but accept that slide export is external and some export features are still roadmap items. ¹⁴¹
- **Best “hackable Markdown IDE with slide mode” (with security caveats):** Yank Note for trusted, code-centric vaults only. ⁷

¹ ⁹ ¹³ ⁴³ ⁵⁰ ¹²⁹ <https://github.com/marp-team/marp-cli>
<https://github.com/marp-team/marp-cli>

2 10 16 55 56 61 136 137 <https://github.com/logseq/logseq>
<https://github.com/logseq/logseq>

3 8 12 42 53 128 133 138 **GitHub - glushchenko/fsnotes: Notes manager for macOS/iOS**
<https://github.com/glushchenko/fsnotes>

4 79 89 <https://docs.zettlr.com/en/getting-started/setup/>
<https://docs.zettlr.com/en/getting-started/setup/>

5 15 29 92 94 98 140 <https://www.qownnotes.org/>
<https://www.qownnotes.org/>

6 37 103 105 141 <https://www.tangentnotes.com/>
<https://www.tangentnotes.com/>

7 39 40 41 <https://github.com/purocean/yn>
<https://github.com/purocean/yn>

11 17 54 57 72 **Logseq - App Store - Apple**
https://apps.apple.com/gb/app/logseq/id1601013908?utm_source=chatgpt.com

14 48 119 121 <https://marp.app/>
<https://marp.app/>

18 62 70 134 **Presentation Mode improvements: Suggestions**
https://discuss.logseq.com/t/presentation-mode-improvements-suggestions/1549?utm_source=chatgpt.com

19 69 **Plugin Configuration | Logseq Slide Reveal**
https://www.vipzhicheng.com/logseq-plugin-slide-reveal/guide/usage/settings.html?utm_source=chatgpt.com

20 74 130 <https://actions.getdrafts.com/a/239>
<https://actions.getdrafts.com/a/239>

21 36 106 108 <https://www.tangentnotes.com/Features>
<https://www.tangentnotes.com/Features>

22 <https://github.com/Zettlr/Zettlr/blob/develop/LICENSE>
<https://github.com/Zettlr/Zettlr/blob/develop/LICENSE>

23 27 78 80 81 <https://lwn.net/Articles/984502/>
<https://lwn.net/Articles/984502/>

24 82 90 <https://docs.zettlr.com/en/guides/presentations/>
<https://docs.zettlr.com/en/guides/presentations/>

25 26 84 87 135 139 <https://docs.zettlr.com/en/export/defaults-files/>
<https://docs.zettlr.com/en/export/defaults-files/>

28 **QOwnNotes**
https://en.wikipedia.org/wiki/QOwnNotes?utm_source=chatgpt.com

30 31 32 33 95 132 <https://www.qownnotes.org/scripting/>
<https://www.qownnotes.org/scripting/>

34 104 109 <https://github.com/suchnsuch/Tangent>
<https://github.com/suchnsuch/Tangent>

35 111 <https://www.tangentnotes.com/Roadmap>
<https://www.tangentnotes.com/Roadmap>

38 47 51 86 88 99 110 **Templates**
https://pandoc.org/demo/example33/6-templates.html?utm_source=chatgpt.com

44 122 <https://quarto.org/docs/presentations/>
<https://quarto.org/docs/presentations/>

45 71 85 115 <https://pandoc.org/demo/example33/10-slide-shows.html>
<https://pandoc.org/demo/example33/10-slide-shows.html>

46 <https://www.deckset.com/>
<https://www.deckset.com/>

49 <https://docs.deckset.com/English.lproj/Customization/02-custom-theming.html>
<https://docs.deckset.com/English.lproj/Customization/02-custom-theming.html>

52 124 <https://quarto.org/docs/presentations/powerpoint.html>
<https://quarto.org/docs/presentations/powerpoint.html>

58 Logseq Marketplace Plugins
https://logseq.github.io/marketplace/?utm_source=chatgpt.com

59 <https://discuss.logseq.com/t/how-to-sync-your-logseq-graph-across-devices/8462>
<https://discuss.logseq.com/t/how-to-sync-your-logseq-graph-across-devices/8462>

60 77 <https://discuss.logseq.com/t/syncing-logseq-db/31285>
<https://discuss.logseq.com/t/syncing-logseq-db/31285>

63 reveal.js (presentation / slide) bugs · Issue #4799 · logseq ...
https://github.com/logseq/logseq/issues/4799?utm_source=chatgpt.com

64 Patching up the Presentation Mode - Look what I built
https://discuss.logseq.com/t/patching-up-the-presentation-mode/30463?utm_source=chatgpt.com

65 Quick Start | Logseq Slide Reveal
https://www.vipzhicheng.com/logseq-plugin-slide-reveal/guide/getting-started.html?utm_source=chatgpt.com

66 vipzhicheng/logseq-plugin-slide-reveal
https://github.com/vipzhicheng/logseq-plugin-slide-reveal?utm_source=chatgpt.com

67 <https://github.com/benjypng/logseq-pandoc-plugin>
<https://github.com/benjypng/logseq-pandoc-plugin>

68 76 <https://discuss.logseq.com/t/pandoc-plugin-for-exporting-export-to-pdf/1247/31>
<https://discuss.logseq.com/t/pandoc-plugin-for-exporting-export-to-pdf/1247/31>

73 131 <https://plugins-doc.logseq.com/>
<https://plugins-doc.logseq.com/>

75 <https://github.com/ergut/mcp-logseq>
<https://github.com/ergut/mcp-logseq>

83 <https://www.zettlr.com/>
<https://www.zettlr.com/>

91 107 pptx corrupt when using with pandoc template. ...
https://github.com/jgm/pandoc/issues/10085?utm_source=chatgpt.com

93 100 <https://www.qownnotes.org/installation/macros.html>
<https://www.qownnotes.org/installation/macros.html>

96 <https://github.com/qownnotes/scripts>
<https://github.com/qownnotes/scripts>

97 <https://www.qownnotes.org/scripting/hooks.html>
<https://www.qownnotes.org/scripting/hooks.html>

101 <https://formulae.brew.sh/cask/qownnotes>

<https://formulae.brew.sh/cask/qownnotes>

102 <https://www.qownnotes.org/scripting/methods-and-objects.html>

<https://www.qownnotes.org/scripting/methods-and-objects.html>

112 **Yank Note: A highly extensible Markdown editor.**

https://yank-note.com/?utm_source=chatgpt.com

113 **Plugins**

https://revealjs.com/plugins/?utm_source=chatgpt.com

114 116 **The HTML presentation framework | reveal.js**

https://revealjs.com/?utm_source=chatgpt.com

117 **Export reveal.js presentation to .odp or .ppt presentation**

https://github.com/hakimel/reveal.js/issues/1702?utm_source=chatgpt.com

118 **Speaker View**

https://revealjs.com/speaker-view/?utm_source=chatgpt.com

120 <https://marketplace.visualstudio.com/items?itemName=marp-team.marp-vscode>

<https://marketplace.visualstudio.com/items?itemName=marp-team.marp-vscode>

123 125 <https://quarto.org/docs/reference/formats/presentations/pptx.html>

<https://quarto.org/docs/reference/formats/presentations/pptx.html>

126 <https://github.com/mpas/deckset-themes>

<https://github.com/mpas/deckset-themes>

127 <https://github.com/gnab/remark>

<https://github.com/gnab/remark>